Spend the Next Token Where Verification Cannot Reach

Train-consistent errors, independent re-derivation, and token budgeting for an LLM program-synthesis agent on ARC-AGI-2

Technical report · Team _hans · November 2026

ARC Prize 2026 Paper Track · Kaggle notebook deepdreamer/fnext-w4a16-arc-submit · submission 57850872 (v33)

Abstract. We built a program-synthesis agent around one open-weight model, Qwen3.8-Flash-Next (125B parameters, 6B active), and ran it on Kaggle's four L4 GPUs for ARC-AGI-2. The agent writes Python in a persistent interpreter and may only submit a program that reproduces every training pair. Serving work took throughput from 120 to over 700 generated tokens per second, and for a long time more tokens per task meant a higher score. Then it stopped: with the harness fixed, doubling throughput moved the public leaderboard from 69.2 to 70.1. We trace the plateau to *train-consistent wrong* (TCW) answers, programs that fit all training pairs but get the test wrong, which account for 60 of 71 lost test inputs in our final offline runs. We argue that no check grounded in the training pairs can expose a TCW answer, that long searches make accidental fits more likely, and that retries from the same context repeat the first attempt's reading. Replays with the first attempt held fixed support this: an independent run rescues 39.5% of doubted wrong answers, against about 20% for a same-chain retry. A token budget built on these findings (floors for hard tasks, then independent second chains) gained 6.1 of 120 points offline at equal tokens and scored 73.89 on the public leaderboard, up from 70.14.

1 Introduction

ARC-AGI-2 asks a system to infer a grid transformation from a handful of input/output pairs and apply it to new test inputs [1, 2]. The Kaggle version adds hard limits: 240 hidden tasks, twelve hours, four 24 GB L4 GPUs, no internet. Two answers are allowed per test input.

Our approach was an LLM agent that solves each task by writing and testing programs. Programs have a useful property for ARC: they can be checked exactly against the training pairs, so the agent never has to guess whether a candidate fits what it was shown. Most of our competition was spent making that agent cheap enough to run at scale on four small GPUs, and for weeks the leaderboard rewarded every gain in throughput.

When it stopped, the remaining errors were not search failures but programs that passed every check we could run and were still wrong; the tokens that helped most went to a second, independent attempt rather than to reviewing the first. Sections 2–7 give the argument and main measurements; the appendices give every experiment behind them. Our contributions:

- A measurement, on the leaderboard and offline, that tokens per task stop buying score while the harness stays fixed, and an account of where the extra tokens went (Section 3).
- A simple argument for why verification against training pairs cannot catch train-consistent errors, why long searches produce more of them, and why same-context retries rarely fix them (Section 4).

- Replay experiments with the first attempt held fixed that compare second sources of evidence, plus the null results that fit the same picture (Section 5).
- A token budget built on these findings, and the serving stack that made a 125B mixture-of-experts model practical on four L4s (Sections 6 and 7).

2 Setting

2.1 Hardware and scoring

A submission runs once over the 240 hidden tasks with twelve hours on four L4 GPUs (24 GB each, about 22 GB usable, PCIe only) and 188 GB of host RAM. Each test input gets two attempts; a task scores the fraction of its test inputs answered exactly. Because generation dominates the run, throughput translates directly into thinking budget: every extra 100 generated tokens per second gives each task about 18k more tokens over the run.

2.2 Model

We serve Intel's W4A16 AutoRound quantization [11] of Qwen3.8-Flash-Next without fine-tuning. The model has 125B parameters with 6B active per token, 512 routed experts per layer (10 active), three Gated DeltaNet linear-attention layers [12] for every full-attention layer, and a 51B-parameter n-gram embedding table (PLE). We tried supervised fine-tuning and GRPO on harness trajectories; none of the fine-tuned models beat the base model on the leaderboard (Section 5.4).

2.3 Agent

The agent uses the tool loop of Tufa Labs' Duck harness [10], with one persistent Python interpreter per task.

- Grids are shown as compact rows of digits. The interpreter offers helpers for connected components, shapes, boundaries and colour and symmetry statistics, which the model calls on demand.
- The model explores with `python(...)`. Every new `transform` is run, in a separate process, on all training pairs, and failures come back as cell-level differences.
- The model answers with `submit(id, issues)`. Only a candidate that passes every training pair is accepted, and the model must list its doubts about the test outputs with a severity from P0 (most serious) to P2.
- A doubted answer starts an attempt-2 phase in the same conversation, which asks for the alternative reading that would change the test output.
- Thinking is capped at 6,144 tokens per turn (8,192 during reviews); sampling temperature is 0.7. The agent writes a complete submission file every five minutes, so a shortened run still produces a valid answer.

2.4 How we measured

The leaderboard allows one submission a day and a single submission has a standard deviation of about 2.7 points, so most decisions were made offline. Offline runs use the 120 public evaluation

tasks under seeded augmentations (two fixed augmented copies), on L40S and H200 machines with a token clock that charges each task what it would cost on Kaggle. Offline scores are optimistic: the expert mask and a short guide of about 20 transformation families were developed on the same public tasks, and for v32 the leaderboard score was 0.86 of the offline one. Single offline runs also differ by several points, so we compare means over several runs where we can and say so where we cannot (Appendix A).

3 More tokens helped, then stopped helping

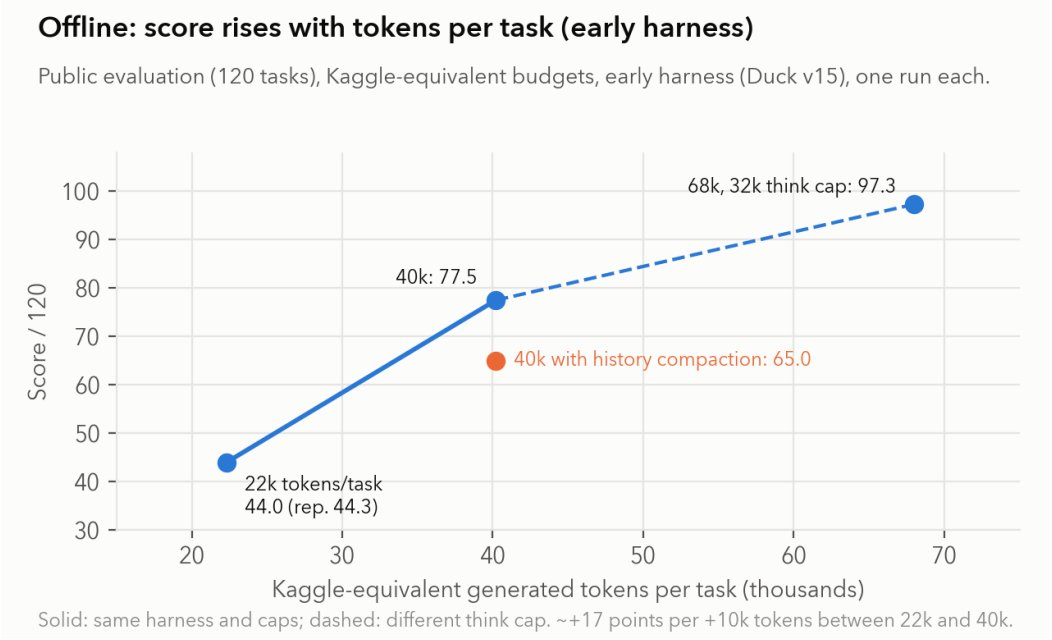


Figure 1. An early version of the harness on public evaluation at three Kaggle-equivalent budgets. Between 22k and 40k tokens per task, score rose by about 17 points per 10k tokens. Compressing old history to fit 40k into a smaller context cost 12.5 points.

3.1 Offline

Early in the competition, score was close to linear in tokens per task (Figure 1). At 22k Kaggle-equivalent tokens per task an early harness scored 44.0 (a repeat run gave 44.3); at 40k it scored 77.5, and the tasks solved at 22k were almost a subset of those solved at 40k. A run with 68k tokens and a larger thinking cap reached 97.3.

3.2 On the leaderboard

Figure 2 shows the leaderboard history (all submissions in Appendix K). Pruning, more lanes and a maturing harness took v1 (22.92) to v13 (mean 69.2 over four submissions). From v13 to v32 we kept the harness fixed and worked only on serving, which took throughput from 315 to 462–639 tokens per second. The score stayed around 70 (v32 mean of three: 70.14).

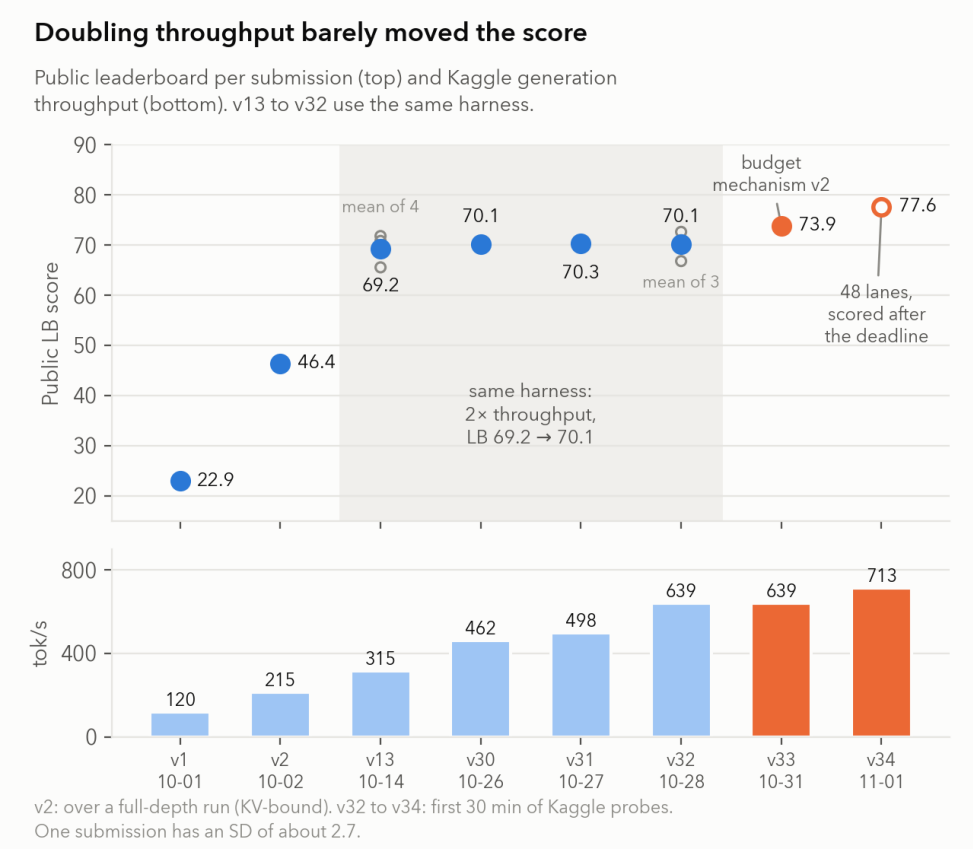


Figure 2. Public leaderboard per submission (top) and generation throughput on Kaggle (bottom). Inside the shaded range the harness did not change; doubling throughput moved the mean from 69.2 to 70.1. v33 introduced the budget mechanism of Section 6.

3.3 Where the extra tokens went

Cut early, idle late. Our time bank split each finished task's leftover time among all running and queued tasks. Replaying it on 240 tasks showed that it cut 41 to 49 hard tasks from the first wave at 100k to 115k tokens, short of the 127k to 137k they needed, while tasks that started late received 175k to 560k tokens they never used, and the run ended 0.5 to 2.4 hours early. The extra throughput mostly went to tasks that did not need it.

Long searches overfit. Across nine offline runs (1,044 first fits), the first train-passing program in a chain was right about 90% of the time when it came within 10k generated tokens, and only 17 to 29% of the time when it came after 80k (Figure 4, middle). In the three runs of the final harness the numbers are 94% (31 of 33) and 29% (12 of 42). Harder tasks take longer, so difficulty is a confound (52 of the 126 wrong first fits in the final runs still ended in a solved task), but the next section argues the direction is expected.

4 Why checking cannot catch train-consistent errors

We call an answer *train-consistent wrong* (TCW) if its program reproduces every training output but not the test output. In our final harness TCW answers are the largest loss class: 15 to 23 of

120 points per offline run, and 60 of the 71 test inputs lost in two offline runs of the final budget (Figure 4). In 61% of lost test inputs the agent had already said it doubted its answer.

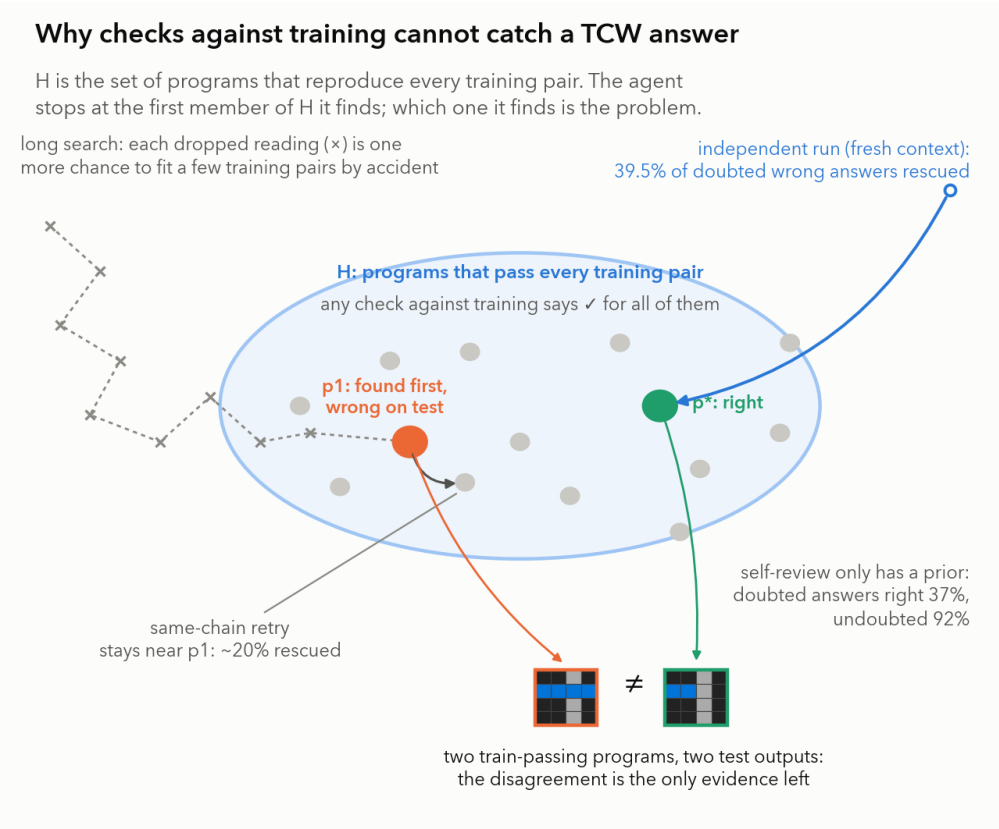


Figure 3. The version-space view. Every program in H passes every training check, so the only remaining evidence is a second member of H that disagrees on the test input.

Let $T = \{(x_i, y_i)\}$ be the training pairs and x^* a test input, and let

$$H(T) = \{\, p : p(x_i) = y_i \text{ for all } i \,\}$$

be the programs consistent with training. The agent searches until it finds a member of H and submits it. Three consequences follow (Figure 3).

1. A blind spot. Any check whose reference is a training output gives the same verdict for every $p \in H$, so it cannot separate the right program from a TCW one. This holds however the check is phrased: re-running the program, asking the model to re-verify cell by cell, or asking a critic to compare outputs with the training targets. What self-review has left is a prior: how natural the program looks, and whether the test input contains a situation training never shows. The prior carries information (answers the agent doubted were right 37% of the time, undoubted ones 92%), but it tells the agent that something is wrong, not what is right. Checks that add assumptions, such as symmetry or invariance audits, are priors too; ours flagged only 8 to 13% of the errors.

2. Search manufactures fits. Each reading the agent tries and drops is another hypothesis tested against a few training pairs. If a wrong reading fits all of them by accident with some small probability q, then the chance that at least one of k wrong readings fits is $1 - (1 - q)^k$, which grows

with k. A program that only appears after a long search is therefore more likely to be one of these accidents. This is the multiple-comparisons problem with the training pairs playing the role of the test set, and it matches the falling curve in the middle of Figure 4.

3. Correlated retries. With two attempts,

$$P(\text{solved}) = P(A_1 \text{ right}) + P(A_1 \text{ wrong}) \cdot P(A_2 \text{ right} \mid A_1 \text{ wrong}),$$

so a second attempt is worth exactly its conditional success rate given that the first failed. A retry in the same conversation inherits the first reading: its alternative is usually another wrong reading, or it changes a spot that was never the problem. Wrong programs are also not close to the right one: across 34 never-solved task-runs, no single-point change to a wrong program produced the correct output. An independent derivation, started from a fresh context, shares fewer of these mistakes. When it also lands in H but gives a different test output, the disagreement is observable without the answer key: it shows the task is ambiguous and supplies the alternative.

So once a train-passing program exists, the only evidence left in the training pairs is disagreement with a second, independently derived member of H, and a token is worth spending only if it can produce one. None of this is new as theory. It is the version-space view of induction [3], the disagreement principle behind query-by-committee [4], and it agrees with the finding that language models rarely correct their own reasoning without outside feedback [5]. Program-synthesis systems have long used behaviour on extra inputs to choose among candidates [8, 9], and self-consistency uses agreement among samples [6]. What we add is a measurement of how this plays out inside an ARC agent under a fixed compute budget, and a budget rule built on it.

5 What rescues a doubted wrong answer

5.1 Replay with the first attempt held fixed

Comparing second-phase strategies by re-running whole tasks is too noisy: the first attempt changes from run to run, and two runs of the same harness differ by more than the effects we want to see. Instead we replayed the recorded first attempt exactly, charged its recorded tokens to the clock, and ran only the second phase under each strategy. We used the 168 states from eight source runs where the first attempt was doubted and wrong.

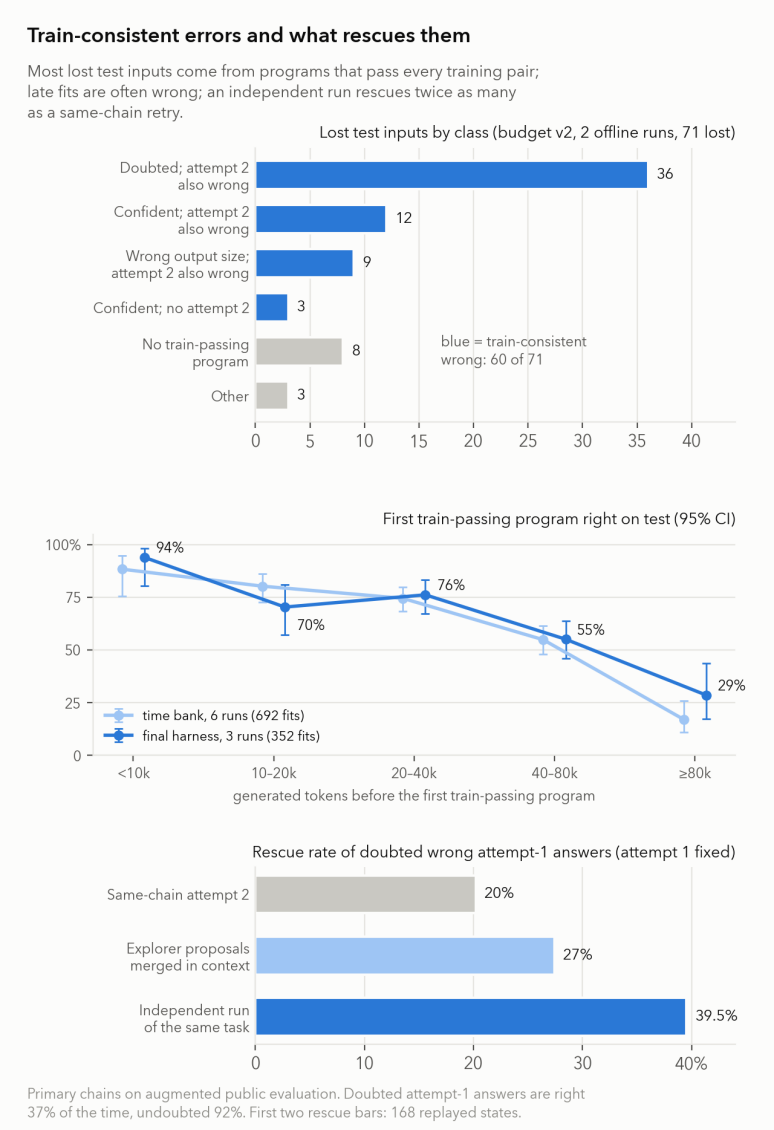


Figure 4. Top: lost test inputs by class in two offline runs of the final budget; blue classes are train-consistent wrong. Middle: share of first train-passing programs that are right on the test input, by generated tokens before they were found (primary chains, 95% intervals). Bottom: rescue rate of doubted wrong first attempts under each second source of evidence.

Second source of evidence	Result on doubted wrong answers
Same-chain attempt 2 (production)	34 of 168 rescued (20%)
Explorer proposals merged into the same context	46 of 168 rescued (27%)
Independent run of the same task[a]	39.5% rescued
Symmetry and invariant audits	flag only 8 to 13%

Table 1. Second sources of evidence for doubted wrong first attempts (168 states from eight runs). First two rows: replay with attempt 1 held fixed. [a]Share of these states whose task was solved by another independent run of the same configuration.

Explorers helped most when they proposed a concrete alternative output rather than describing a doubt. For independent runs the rescue rate was 44.5% when the first attempt had one doubt line and 27% with two or more; 35% of these states were never solved by any run.

5.2 Two independent runs as the two attempts

On the original evaluation tasks, using two independent full runs of the same configuration as attempt 1 and attempt 2 scored 93.2, against 80.6 for a single run. Of that gain, 92% came from tasks where both runs passed training but produced different test outputs, which is exactly the disagreement signal of Section 4. Taking the second answer from the first run's own checked candidates gained at most one point, because a chain stops at its first pass and rarely holds a second distinct one.

5.3 Depth matters

On 45 hard tasks, a fresh second chain with 64k tokens added nothing, while one with 128k tokens added 3.3 points per run. Second chains that started on tasks with no train-passing program at all rescued 0 of 8. A second chain helps when it can reach H on its own.

5.4 Null results that fit the same picture

Many ideas that added text to the same conversation did not help: idea boards, task fact sheets, residual-error reviews, critics and advisors. Table 2 lists the ones with a clean comparison.

Change	Result
Fresh restart when no near-pass by 43k tokens (augD1, one run)	79.8 vs 96.7; 0 of 33 restarts rescued
Thinking cap raised to 32k at the same budget (augD1, one run)	90.8 vs 96.7
Thinking cap 12k instead of 6k	-1.6
Model chooses when to stop iterating (augD1, one run)	94.3 vs 96.7; it almost never iterated
Dihedral/equivariance post-processing	13% recall, voting gain about 0
SFT and GRPO on evaluation-derived tasks	leaderboard 63.5 to 71.4 vs base mean 69.2

Table 2. Changes that did not help. Offline single runs differ by a few points, so only the large gaps are clear.

6 Budget mechanism v2

The final harness spends tokens with these findings in mind (Figure 5):

1. Every task gets a base budget of 86k generated tokens on Kaggle's clock. Whatever a task leaves goes into a shared pool.
2. A task is marked hard if its first answer carries a P0 or P1 doubt, or if nothing passes training by 60k tokens. Hard tasks get a floor of 160k tokens from the pool, raised to 192k when they get close, so they are no longer cut off in the middle of a search.
3. After the main pass, a tail queue runs blind second chains: fresh conversations that stop at their

first train-passing program, whose answer replaces attempt 2. Doubted tasks with long first chains go first; tasks with nothing train-passing go last. Leftover wall time near the end also funds chains.

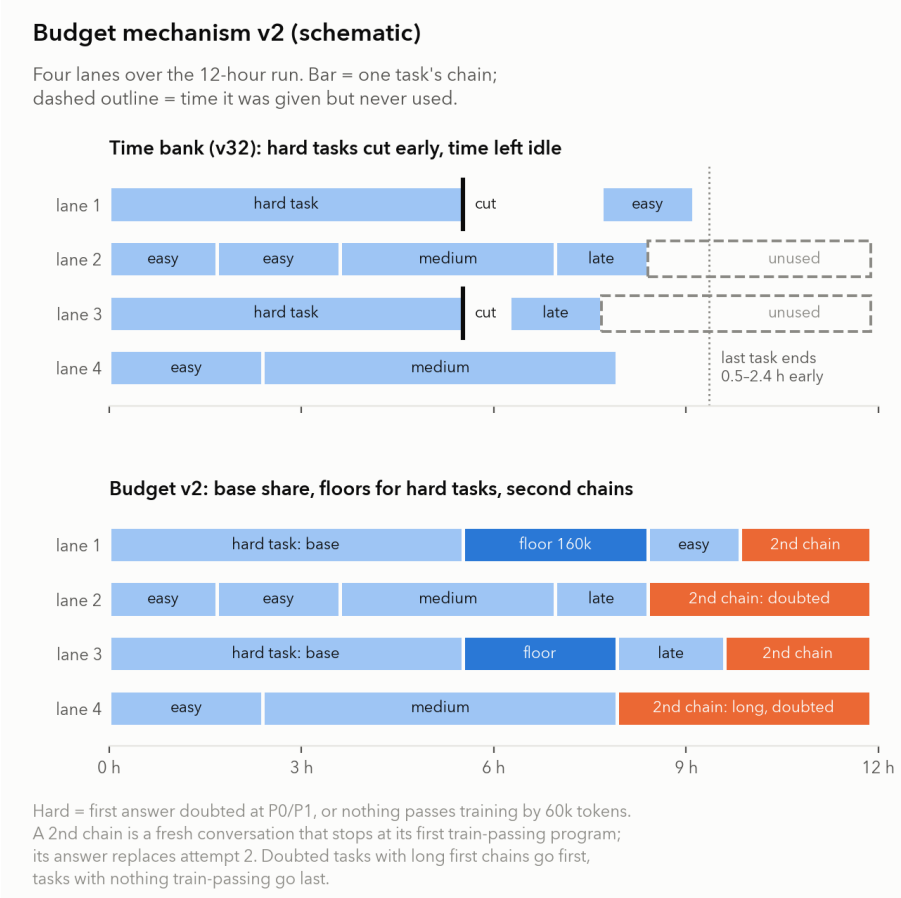


Figure 5. Schematic of the time bank and of budget v2 over a 12-hour run (four lanes shown; the real run has 32 to 48).

At equal total tokens (about 9M per 120-task run on augmented public evaluation, four runs each), budget v2 scored 98.5 against 92.4 for the time bank. Most of the gain came from the floors; the tail chains added 1 to 3 points. On the leaderboard v33 scored 73.89, against the v32 mean of 70.14. The same harness with 48 lanes (v34) scored 77.64, but only after the deadline.

7 Serving a 125B mixture of experts on four L4s

Served naively, the weights took 18.6 of 22 GB on each GPU, which left a KV pool of 0.11M tokens: two conversations at 120 tokens per second. Figure 6 shows the steps that changed this.

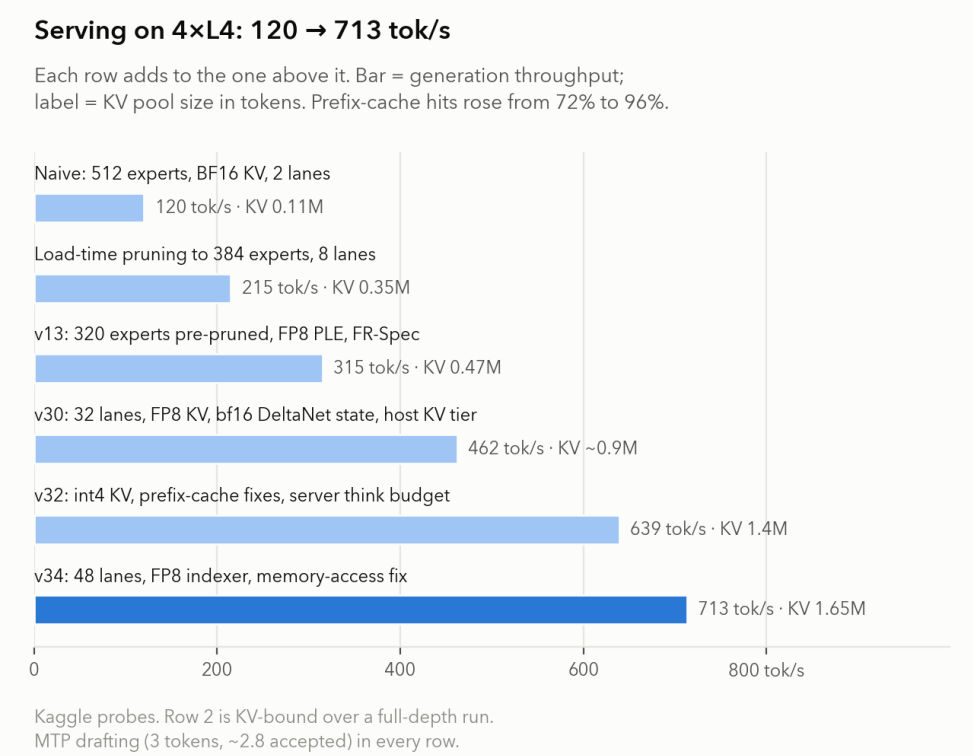


Figure 6. Serving steps on Kaggle's four L4s, each adding to the one above. Prefix-cache hits rose from 72% to 96% over the same steps.

Expert pruning by routing. We counted which experts ARC traffic actually routes to. The final mask keeps 320 of 512 experts per layer: it starts from a public REAP keep-set of 384 [13], swaps in 32 experts that ARC traffic uses, and drops the 64 least used. Keeping more experts did not help: scores with 512, 448, 384, 352 and 320 experts were within noise, and on our 45 hard tasks 512 experts scored below 320 (25.8 against 28.3). Pruning grew the KV pool fourfold, so eight conversations fit (Appendix C).

Prefix caching on a hybrid model. A linear-attention state cannot be rewound, so the server can reuse a conversation's cache only if the new request extends the old one exactly. Three bugs broke this: the harness normalised blank lines in stored replies, the speculative drafter dropped a block on every hit, and decoded linear-attention states were never cached. Fixing them raised the hit rate from 72% to 96%. Because any edit in the middle of the history forces a full re-prefill, the harness edits history only when it must truncate.

Memory. FP8 copies of the n-gram embedding table pinned in host RAM, an INT4 KV cache, bf16 DeltaNet state and an 84 GiB host-memory KV tier give a pool of 1.49M tokens in v32, and 1.65M with an FP8 attention indexer added for v34. The model runs tensor-parallel across all four GPUs with expert parallelism, because the 640-wide experts do not split into 128-wide quantization groups under four-way tensor parallelism.

Decoding. An INT4 multi-token-prediction drafter [14, 15] proposes three tokens per step over a 16k-token FR-Spec vocabulary [16], and about 2.8 are accepted; the reduced vocabulary alone added 6 to 10% throughput. The server enforces the thinking budget itself, so the harness never sends continuation requests, and a watchdog restarts the server and resumes the agents if it dies.

We serve with a patched vLLM 0.30 [17].

8 Discussion

Related work and what is new. Sampling many candidate programs and filtering them by the training pairs is an established route on ARC: Greenblatt [7] sampled thousands of Python programs per ARC-AGI-1 task and voted among those that fit. AlphaCode [8] and CodeT [9] choose among programs by clustering or ranking them on their outputs for extra inputs, and self-consistency [6] votes over sampled answers. These methods spend one large, uniform sample on every task, and they treat agreement as a selection rule. Our setting differs in two ways that change the design. First, one agent run costs tens of thousands of tokens and the Kaggle budget is about 86k per task, so a task gets one or two runs, not thousands of samples, and the question is which task gets the next run. Second, our agent reports its own doubts. We contribute (1) a measurement of where the remaining errors are once search succeeds (60 of 71 lost test inputs are train-consistent), of how first-fit accuracy falls with search length, and of which second attempt rescues a doubted wrong answer (independent run 39.5%, same-chain retry 20%, explorer merge 27%, with the first attempt held fixed); and (2) a budget that follows these measurements: floors so long searches finish, then independent attempts for doubted tasks, those with long first chains first. At equal total tokens this beats the time bank by 6.1 points offline.

Beyond ARC. TCW errors are the ARC version of code that passes the visible tests and fails the hidden ones, a central failure of coding agents. The same blind spot exists wherever the verifier is a handful of examples. The budget rule carries over: fund search until a candidate passes the verifier, with floors so hard instances are not cut off midway; after that, spend on independent candidates rather than on reviewing the first, and use their disagreement both as the doubt signal and as the second answer. The serving lessons apply to any long multi-turn agent on small GPUs.

Limits. Each leaderboard number is a single submission (SD about 2.7), offline numbers are optimistic (Section 2), and several null results in Table 2 are single runs. The replay experiments measure the second phase only.

9 Conclusion

Serving speed bought score until the remaining errors were programs that every available check accepted. After that, the next token was worth most in an independent derivation that can disagree with the first, and a budget built on that gave our best leaderboard result.

Reproducibility

The submission notebook deepdreamer/fnext-w4a16-arc-submit contains the full harness and serving code; version 34 of the notebook is submission 57850872 (v33 in this paper). Appendix K lists every scored submission. This report, its figures and the script that draws them are in the repository `sdmhans/arc-agi-2-flash-next-agent` on GitHub.

Acknowledgements

We thank Tufa Labs for the Duck harness, Intel for the AutoRound weights, dfranzen for the Kaggle reshard, albucino for the INT4 MTP drafter, and the authors of REAP and FR-Spec. Much of our code and this paper was written with help from Claude (Claude Code); all numbers come from our own runs.

References

[1] F. Chollet. On the measure of intelligence. arXiv:1911.01547, 2019.

[2] F. Chollet, M. Knoop, G. Kamradt, B. Landers, H. Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems. arXiv:2505.11831, 2025.

[3] T. M. Mitchell. Generalization as search. *Artificial Intelligence*, 18(2):203–226, 1982.

[4] H. S. Seung, M. Opper, H. Sompolinsky. Query by committee. In *COLT*, 1992.

[5] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, D. Zhou. Large language models cannot self-correct reasoning yet. In *ICLR*, 2024.

[6] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, D. Zhou. Self-consistency improves chain of thought reasoning in language models. In *ICLR*, 2023.

[7] R. Greenblatt. Getting 50% (SoTA) on ARC-AGI with GPT-4o. Redwood Research blog, June 2024.

[8] Y. Li et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

[9] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, W. Chen. CodeT: Code generation with generated tests. In *ICLR*, 2023.

[10] Tufa Labs. The Duck harness. ARC Prize 2026 ARC-AGI-3 solution writeup, Kaggle, 2026.

[11] W. Cheng et al. Optimize weight rounding via signed gradient descent for the quantization of LLMs. arXiv:2309.05516, 2023.

[12] S. Yang, J. Kautz, A. Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *ICLR*, 2025.

[13] M. Lasby et al. REAP the experts: Why pruning prevails for one-shot MoE compression. arXiv, 2025.

[14] F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, G. Synnaeve. Better & faster large language models via multi-token prediction. In *ICML*, 2024.

[15] Y. Leviathan, M. Kalman, Y. Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

[16] W. Zhao et al. FR-Spec: Accelerating large-vocabulary language models via frequency-ranked speculative sampling. In *ACL*, 2025.

[17] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

Appendices

A Evaluation protocol

A.1 Datasets

- **eval120**: the 120 public ARC-AGI-2 evaluation tasks. Until 4 October we made decisions on these originals.
- **augD1, augD2**: two fixed augmented copies of eval120. Each task gets a seeded random non-identity dihedral transform and a permutation of colours 1–9; answers are mapped back before scoring. From 4 October all decisions used these. Augmentation cost the 87 configuration about 11% (aug mean 75.9 against 85.6 on the originals), almost entirely as more train-passing but test-wrong answers rather than lost geometry. The two seeds are not equally hard: augD2 scores run higher than augD1.
- **Hard benches**: 45 "online-hard" augD1 tasks (hard in at least two of three runs, solved with fresh fixed-budget chains), and later a 120-task hard set selected from screen-zero tasks. Early screening sets (dev72, dev26) proved unreliable and are only used here for mechanism notes.
- **Instability**: across 18 augD1 runs, attempt 1 was always right on 32 tasks, never on 13 and sometimes on 75. Across 13 runs of another configuration the per-run mean was 87.4 while the union reached 111.3.

A.2 Kaggle-equivalent budgets

Offline runs must give each task what it would get on Kaggle. We first converted wall time between machines (for example, an L40S/H200 throughput ratio of 2.44 without MTP). This drifted with CPU contention: time-only H200 evals gave 41–52k tokens per task for the same seconds. From mid-October we anchored on tokens measured on Kaggle itself: a probe of the v104 configuration generated 45.3k tokens per task, and every offline budget was rescaled to that anchor. For the final harness the agent runs on a *token clock*: a bank of 86k generated tokens per task, with the harness's time-based triggers driven by tokens instead of seconds (otherwise triggers such as "skip attempt 2 if early" fired at 38k tokens instead of 21.5k on faster machines). Kaggle v33 and v34 use the same token clock, about 20M generated tokens per 240-task run.

A.3 Noise

Setting	Spread
Public leaderboard, v13 submitted four times	71.81, 68.75, 65.56, 70.83 (SD about 2.7)
Public leaderboard, v32 submitted three times	70.97, 72.64, 66.81 (SD about 3.0)
Standard eval120, swap32-cut320 at a fixed budget, 6 runs	74.0 to 84.5 (mean 80.5)
Standard eval120, the 87 configuration, 3 runs	87.17, 86.83, 82.83
Augmented, same seed	40–51 of 120 tasks flip between runs; observed SD 5.1–5.5
Final-era multi-wave runs on augD1, same configuration	99.50 vs 93.50 (24 tasks differ)
45 hard tasks, fixed budget	SD about 3 tasks per run

Table 3. Run-to-run noise. Because of it we required several runs per arm for small effects, and treated single-run differences under about 5 points as noise.

A.4 Leaderboard versus offline

For v32, the leaderboard score was 0.864 of the matching offline score (four offline runs of the same harness, mean 97.46 of 120).

B Resource curves

B.1 Score versus tokens per task

Configuration (standard eval120, one run unless noted)	Tokens/task	Score
Early harness, 195 s per task	∼17k	31.0
Early harness, 276 s per task	∼24k	41.5
KT20 (two runs)	22.3k	44.0, 44.33
KT40	40.2k	77.5
KT40 with history compaction to fit a 32k context	40k	65.0
U0: think cap 32k, 192k context	68k	97.33
87 configuration, bank 245 s vs 530 s	24k vs 52k	51.0 vs 85.6
V104 on augD1 (L40S): bank 820 vs later 1020-equivalent stack	47.4k vs 56.2k	78.83 vs 91.83
V104 with decode cache on (+8% tokens, augD1)	51.4k	87.33 (from 78.83)

Table 4. Tokens per task and score. The two KT20 runs solved 48 and 47 tasks, 32 in both and 63 in either; their union (59 fully solved) was still far below KT40's 82, so extra throughput was better spent on longer chains than on more chains.

Hard tasks at fixed budgets. On the 45 hard augD1 tasks, a single fresh chain capped at 128k, 160k and 192k scored 25.3, 27.9 and 30.0 on average over two runs, using 98k, 104k and 113k tokens. Two independent chains of 96–128k each scored about 31 using 179–203k. Only 1–5 of the 45 tasks reached a 192k cap.

Water-filling. Simulations on the U0 run showed why allocation matters: fixed per-task caps of

17k/24k/48k would have scored 12/27/60.5, an oracle shared pool 68.5/80/93.8, but uninformed water-filling at 20k/24k only 25/31.5. Correct answers arrive late: the U0 tokens-to-correct quartiles were 23k/40k/66k (p90 107k).

B.2 Thinking caps

Experiment	Result
V104 era (augD1+augD2 means), 6144 / 8192 / 12288	81.6 / 82.8 / 82.6 (no effect)
Final era, 12288 (reviews 16384) vs 6144 (8192)	augD1 95.17 vs 98.0, augD2 97.50 vs 97.83 (about -1.6)
Final era, 32768 at the same 86k budget	90.83 vs 96.67; tasks without any train-passing program 12 vs 2
45 hard tasks, 8k cap	28.33 (+0.00)

Table 5. Thinking caps. Short caps force more tool rounds; at fixed budgets, more Python tries beat longer thinking. In production about 24.5% of rounds and 53% of tokens hit the 6,144 cap.

B.3 Lanes, temperature

Throughput on the four L4s first saturated near 300–310 tokens per second from eight lanes on, and expert count did not change that ceiling (384 or 320 experts both reached about 302); the extra lanes only became useful once the KV pool and host tier grew (Appendix D). On real agent traffic, 48 lanes gave 836 tokens per second against 736 at 32 in a lane-gate probe, with per-lane speed falling from 23.0 to 17.4. On a longer probe, throughput fell from 816 with 47 requests running to 493 with 30 once the KV pool filled. Temperature 0.7 beat 1.0 on an early screening set (36.0 vs 32.0); 0.2–0.5 were within noise.

C Model and pruning

The model has 48 layers in twelve groups of three Gated DeltaNet layers and one sparse-attention layer, 512 experts with 10 routed per token, and a 51B-parameter n-gram table. The INT4 checkpoint is 181 GB (routed experts 62.8 GB, n-gram table 102.5 GB in BF16, MTP head 5.2 GB). Because the expert width of 640 splits into 160 under four-way tensor parallelism, which is not a multiple of the 128-wide quantization group, we serve with expert parallelism.

Mask (standard eval120 unless noted)	Score
Unpruned 512 experts, bank 500	85.33
ARC-448 (top routing mass on ARC traffic)	86.83
REAP-384, bank 500 (3 runs)	81.0 / 81.5 / 82.5
ARC-384	77.17
swap32-cut320 (REAP-384, 32 swaps toward ARC-heavy experts, cut to 320), 4 runs	mean 82.25
reap-cut320 (no swap step), 3 runs	83.0 / 77.5 / 77.17
swap32-cut288 vs cut320, 87 configuration, bank 530	79.0 (2 runs) vs 85.6
swap32-cut256	62.0
At the Kaggle-realistic budget: REAP-384 x8 lanes (bank 365) vs swap32-cut320 x8	75.7 (2 runs) vs 82.25
45 hard augD1 tasks, 160k: 512 experts vs cut320	25.8 vs 28.3 (2 runs)

Table 6. Expert masks. Our final reading (mid-October, more runs per arm): scores with 512, 448, 384, 352 and 320 experts were within noise, cut288 was about 4 points lower and cut256 far lower; on the hard set 512 experts were below 320. At Kaggle's real throughput the pruned model wins clearly because it fits more lanes. Re-deriving the mask from each fine-tuned model's own routing moved scores by 3 to 9.5 points on the originals: masks are fitted to eval120 traffic.

Quantization and KV. FP8 copies of the n-gram table (from a public NVFP4 checkpoint) cost no score (91.83 vs 90.33 on augD1) and halve its host memory. FP8 KV needed a two-file backport from vLLM; on an L40S server it grew the KV pool 1.57x but cost 7% throughput and lowered draft acceptance, so it only paid on Kaggle once lanes became KV-bound. The final stack uses INT4 KV with the DeltaNet state kept in bf16.

Speculative decoding. The INT4 MTP head with three draft tokens averaged 2.68–2.92 accepted tokens on Kaggle (per-position acceptance 0.77/0.60/0.47). Quantization and pruning did not reduce acceptance (INT4 head 2.88, BF16 head 2.84, unpruned 2.84). Four draft tokens raised acceptance to 3.20 but not throughput. A 16k-token FR-Spec draft vocabulary raised throughput 11.6% on a Kaggle probe (330 vs 296 tokens per second at eight lanes) with no score change (90.33 vs 89.5); 8k and 6k vocabularies gave the same acceptance.

Other base models in the same harness. Qwen3.8-27B FP8 scored 32.0 at about 39k tokens per task and 57.5 at about 67k (standard eval120), and its decode slowed sharply on L4 as contexts grew. DeepSeek-V4-Flash scored 9–10 at about 22k tokens. A dots3 preview model scored 54.0 at 52k against 85.6 for Flash-Next, and the two were barely complementary (union +4 vs +10 for two Flash-Next runs).

D Serving on four L4s

Probe	Configuration	tok/s	KV pool
v3	unpruned, BF16 KV, no MTP (agent replay)	132	110,636
v9	first submission configuration, 2 lanes, MTP k3	120	–
v11	REAP-384 pruned at load time, 8 lanes	∼305	317,499
v12	swap32-cut320, 10 lanes (saturates from 8)	300–340	471,859
v13 bench	REAP-384 x8 on 16 hard tasks at full depth (KV-bound)	215	–
FR-Spec	v104 configuration with 16k draft vocabulary, 8 lanes	330	–
P1	v13 stack, 8 lanes, BF16 KV	315	–
P10 (v30)	32 lanes, FP8 KV, host KV tier, bf16 DeltaNet state	462	–
P12 (v31)	24 lanes, INT4 KV	498	1.26M
P14 (v32)	full v32 stack, 32 lanes	639	1.49M
P15	lane gate: 32 / 40 / 48 lanes on real traffic	736 / 693 / 836	–
P16 (v34 stack)	48 lanes + FP8 attention indexer	713	1.65M

Table 7. Kaggle probes (four L4s). The v34 figure comes from probe P16 of the same stack, not from the scored v34 run, whose logs Kaggle does not show.

Prefix caching. An early replay reused only 2.7% of the prompt. Three causes kept the hit rate at 72% on Kaggle after the obvious fixes: the harness dropped blank lines from stored replies, so the replayed history never matched; the speculative drafter released one 800-token block per hit; and vLLM never cached decoded tokens of the linear-attention state when MTP was on. Fixing the first two gave 84.6%; caching decoded states raised it to about 96% (H200 90.7 to 96.2%, L40S 89.0 to 96.1% with +10% throughput). With 32–48 lanes the GPU cache is too small and an 84 GiB host tier serves about 90% of the misses.

Other gains. Enforcing the thinking budget inside the server removed continuation requests, which had been 45–47% of recomputed prefill. Sharded sampling sped up per-request decode by 5–11%.

Dead ends. Four draft tokens (higher acceptance, no throughput), FP8 two-shot all-reduce (slower than NCCL), FP8 hyper-connection weights (replay -2 to -5%), and FP8 KV at eight lanes on L40S (-7% throughput).

E Harness components and ablations

Baselines change over the competition; each table states its own. Most rows are single runs, so differences under about 5 points are noise (Appendix A).

E.1 Early harness (standard eval120, H200)

Change	Result
Auto-check on all training pairs, cell diffs, grid helpers (KT20 budget)	43.67 vs 44.0/44.33; more train passes (54 vs 47/49) but the extra ones were overfit
Risk audit with a flagged second candidate (KT20)	41.5
Removing a visible "estimate your difficulty" request (screening set)	$+9$
Reviewer showing the test outputs again (a display bug had hidden them since v23), plus test audit: the "87 configuration"	87.17 / 86.83 / 82.83 vs 80.5
Risk check during search instead of after acceptance	83.0 / 81.5 / 81.5 vs 87.17 / 86.83
Extra views of the grid, training-pair augmentation, diff/invariant feedback (87 base)	79.5 / 76.5 / 82.5
Compressing history at about 22k tokens	about -1
Fixed time per task instead of a shared time bank	-14.7 (65.83 vs 80.5)

E.2 Self-review and guides (augmented, H200)

Change	Result
v42: review lists doubts as P0/P1/P2, review thinking 8,192, forced close	82.5 (80.5/84.5) vs 77.5
v49: a doubted answer triggers the alternative-reading attempt 2	84.92 vs 82.5; attempt 2 rescued 10 and 6 tasks but attempt 1 fell 3–4.5
v52 / v53: doubts ranked by consequence / a counterexample-based P0	82.25 / 83.25
Removing the review module and idea book from v49	about -8 on each seed
No review at all vs full review (L40S, 4 runs each)	86.96 vs 90.61
Time-triggered "stuck" guide (bank, two seeds)	75.25 vs 82.5, dropped
Idea book of transformation families in the prompt vs as a tool	82.5 vs 87.33
Helper card added to the system prompt (4 runs each, offline)	90.96 vs 88.62, but leaderboard 65.00 vs 69.31

The helper card's offline gain did not survive the leaderboard; part of it traced to a helper that raised an error inside every `transform` that called it, in all versions from v102 to v109.

E.3 Mechanisms that did not help

Mechanism	Result
Exploration ledger without review (26 hard tasks)	5.0 / 5.0 vs about 9.7; 24 of 26 tasks tried exactly one idea before the first pass
Ideas board (three versions, 8 runs)	consistently -3 on attempt 1
Explore forks / skill review / early challenge	worse / null / 38.5 vs 40.0
Single-point mutations of wrong programs	0 of 34 never-solved task-runs reached the right output
Time-bank overdraft of 15%	within noise offline; leaderboard 69.03 / 65.56
Direct-answer attempt 2	82.33 vs 88.83; ran 39 times, added nothing
Independent branch on doubt (early version)	81.5
Alternative readings written at submit (4–6 runs)	89.96–90.61 vs 90.96
In-context "view" second chain	rescued 0.5 tasks for 38% of a run's tokens

E.4 Final-era harness (L40S, token clock 86k, augD1, baseline 96.67 unless noted)

Mechanism	Result
Fresh restart when no near-pass by 43k	79.83; 0 of 33 restarts rescued, 16 of these tasks the baseline solved late
Independent view switch at a P0/P1 doubt	91.17; rescue after a wrong first fit 8/36 vs 10/36
Model chooses when to stop iterating	94.33; 68 of 72 called finish right after a second version
Review-driven iteration loop	105.33 / 98.83 on augD1, but -6 to -10 on the hard set (17.5/15.0 vs 23.5/25.0)
Critic at attempt 1 (35-task subset)	called 16–19 right outputs wrong, 2–8 wrong outputs consistent
Symmetry cell hints (two variants)	97.67 / 95.17
Invariant audit (offline)	flags 52% of right vs 56% of wrong answers
Context 64k / 48k	13.5 vs 24.25 on the hard set; 89.00 vs 103.17 on augD1
Hint to switch readings at compaction	17.5 vs 24.25 (hard set)
Residual review / close-left nudges (45 hard, 160k)	25.0 / 25.9 vs 28.3
Fresh advisor at a stuck point (52 states)	28.94 vs 27.50
Live advisor in multi-wave runs (5 runs vs 5 controls)	about 0
Reviewer at attempt 1 (76 states)	-1.0 / -2.0
"Let me reconsider" opener	-0.06 per state

A token audit of 90 chains of the v32 harness: model thinking 61% of context, tool output 12%, model code 10%, grids 5.7%, system text 4.5%. Of 26 unsolved tasks, 25 ended with two train-passing but test-wrong candidates, and 73% of their tokens were spent before attempt 1.

F Train-consistent errors and attempt 2

F.1 Loss anatomy

In two runs of budget v2 (augD1 and augD2, 344 test inputs), 71 test inputs were lost: doubted (P0/P1) first attempt with attempt 2 also wrong 36 (51%); confident wrong with attempt 2 wrong 12 (17%); wrong output size with attempt 2 wrong 9 (13%); no train-passing program 8 (11%); confident wrong with no attempt 2, 3; other 3. The first four classes, 60 of 71, are train-consistent wrong. Counting every lost input whose first attempt was doubted, regardless of attempt 2, gives 43 of 71 (61%). For the doubted-and-still-wrong cases: the doubt missed the real error in 21, the named alternative was another wrong reading in 10, the right reading was named but not used in 8, a right attempt 2 was displaced by a tail chain in 2, and a right manual answer was cut by ranking in 2. Over the final family of runs, train-consistent losses were 15 to 23 points per run, against about 1.2 for selection and 2–3.5 for budget cuts (10–12 under the old time bank).

F.2 Self-review calibration

Runs	Right rate of attempt 1
Five final-family multi-wave runs	P1-doubted 37%, undoubted 92% (P0 never used)
Three final-era single-wave runs	doubt-free 82–89%, doubted 35–45%
v42 (218 final submits, 63 wrong)	wrong when last doubt was P1 24/31, P2 33/139, none 2/42
eval120 review audit	P0/P1 catches 62% of wrong answers with 13% false alarms on 510 right ones

Table 8. The review is a useful prior. In v42, the doubt list named exactly the faulty decision or region in 36 of 63 wrong answers (57%), a related one in 15, and missed it in 12. Routing extra budget by review was no better than random per token, however.

F.3 Two independent runs as attempts 1 and 2

On standard eval120 at a fixed budget (H200), using two independent full runs of the same configuration as the two attempts raised the score from 80.6 to 93.2; 92% of the gain came from tasks where both runs passed training with different test outputs. Taking attempt 2 from the same run's other checked candidates gained at most one point, because a chain stops at its first pass. In later offline runs, voting over k independent full runs gave 92.3, 93.5, 96.2, 99.7 and 104.8 for $k = 2$, 3, 5, 10 and 30.

F.4 Replay study (attempt 1 held fixed)

States were taken at the first-attempt submit of eight multi-wave runs, giving 265 states with a P1 doubt, of which 168 had a wrong first attempt. The replay re-sent the recorded first attempt byte for byte and charged its recorded tokens; a control path reproduced the original attempt-2 request exactly (4 rescues in both).

Second-phase strategy	Rescues
Original same-chain attempt 2 (all 168)	34 (20%)
Report-only explorers + merge in the main chain, v1 (all 168)	42
Explorers that must propose concrete cells, v3 (all 168)	46 (27%); +0.070 per state vs original, 90% CI +0.022 to +0.125
v1 or v3 (union)	54
Answer paths on one run (20 states): alternative / structured / third reading / recheck	4 / 5 / 5 / 4
Any of seven paths / merge of the seven by the main chain	6 / 6

Table 9. Replay results. The v3 explorers cost about 75k tokens per flagged task; only 46% of P1 states could fit that inside the 160k floor. Over 228 states with explorer reports, both versions stayed wrong in the same way 70% of the time.

Report audit. An LLM judge compared 233 explorer reports (61 states) with the gold outputs. Of 149 reports that said "keep attempt 1", 121 kept cells that were wrong. Only 13 of 61 states had any report pointing to the fix, and when one did, the merge used it 13 of 14 times: the bottleneck was exploration, not merging.

Independent re-derivation. 39.5% of the 168 wrong doubted states were solved by some other independent run of the same task: 44.5% when the first attempt had one P1 line, 27% with two or more. 35% were never solved by any run.

Checks that add priors. Dihedral/equivariance post-processing recalled 13% of errors and voting with it gained about 0. A contact-region gate lifted precision only 1.3x. Over 4,380 programs, the 284 that our audit lines flagged were right only 1.1% of the time, but the audit caught just 8% of wrong first attempts. Where several versions of an answer disagreed, the cells that were wrong in every version lay outside the region of disagreement (0 of 130 inside), so disagreement localises the ambiguity but not every error.

G Budget mechanism details

G.1 What was wrong with the time bank

The time bank never gave a task less than its base share, and every finished task donated its unused seconds to all running and queued tasks. That made the outcome depend on queue order: in a simulation of the Kaggle run, early tasks got about 1.1x the base and late ones about 3x. The harness's "easy first" order (by grid size) put the historically hardest tasks first (expected solve rate 0.28 against 0.53 for the last quarter). Replaying the exact time-bank code on 240 tasks showed the run ending 0.5–2.4 hours early while 41–49 hard first-wave tasks were cut at 100–115k tokens, against the 127–137k they needed; a value model put the cost of those cuts at no more than 7–12 points per 240 tasks (about 3–5 leaderboard points). On our machines, multi-wave runs under the time bank scored 4–6 below single-wave runs of the same harness, which is the same effect.

G.2 Design of budget v2

- Base 86k generated tokens per task; unused tokens go to a shared pool.
- A task is flagged hard when its first answer carries a P0/P1 doubt or nothing passes training by 60k tokens. In practice 52–55 of 120 tasks were flagged (10–13 for a doubt, 42 for no pass), and flagged tasks used 111–115k on average.
- Flagged tasks get a floor of 160k from the pool, extended lazily to 192k when they come within 10k of their allotment. The values came from the hard-task budget curve (Appendix B).
- A tail queue runs second chains capped at 160k (at least 100k needed to start): fresh conversations that stop at their first train-passing program and replace attempt 2. Order: doubted tasks with long first chains first; tasks without any train-passing first attempt last.
- On Kaggle the deadline of every task is the global end of the run, tokens set the budgets, and leftover wall time near the end also funds chains.

G.3 Results

Run (multi-wave, 120 tasks, 24 lanes, 86k per task)	augD1	augD2	Tokens
Time bank, run 1	94.17	91.33	9.24M / 8.81M
Time bank, run 2	91.00	93.17	–
Budget v2 on the v32 harness (primary only)	98.83 (97.83)	97.17 (94.17)	9.06M / 8.96M
Budget v2, exact v33 build	99.50	98.67	–
Budget v2, v33 build, repeat	93.50	94.83	–

Table 10. Budget v2 against the time bank. Four-run means: 98.54 against 92.42 (+6.1). Including the two later repeats of the v33 build, the budget-v2 mean over six runs is 97.1. On the leaderboard v33 scored 73.89 against the v32 mean of 70.14.

G.4 Tail chains

- Value depends on whom they go to. In a selection study of 175 tail chains, tasks with no train-passing first attempt got 0 rescues from 8 chains of 120k each; tasks with a doubted attempt 1 and a primary of at least 100k gave the best return, 5.0 points from 41 chains (1.33 per million tokens).
- Length matters: a chain cut at 60k kept 18–26% of a full chain's value, 85–90% at 120k. On 45 hard tasks a 64k second chain was worth about nothing, a 128k chain 3.3 points per run.
- With a realistic pool (about 11k spare tokens per task on Kaggle), the order decides whether tail chains pay: with the old order the net was about 0, hard-first made it positive.
- Merge rule: replacing attempt 2 scored 31.5 on 57 tail task-runs, against 29.0 for filling only an empty slot, 28.0 per test input and 26.0 for keeping the primary. Weighted voting lost on held-out runs (-1.0 per run).

H First-fit analysis

This appendix gives the data behind the middle panel of Figure 4 and the "long searches overfit" claim. It was computed from the logs of existing offline runs on augmented public evaluation, without new generation.

Definitions. A *chain* is one conversation: the primary chain of a task, or a tail second chain. Its *first fit* is the earliest program the harness checked that passed every training pair, whether or not it was submitted. *Tokens at first fit* are the generated (completion) tokens of that chain up to and including the reply that wrote the program; this is the same token clock the budget uses. A first fit is *right* if all of the task's test outputs match the gold exactly (about 40% of fits are on tasks with two or three test inputs). *Task solved* means the task was finally scored correct by the run, which can happen even when the first fit was wrong.

Runs. Final harness (budget v2 on the v33 harness): ev3-d1-mwv33 (augD1, 99.50), ev3-d1-mwv33r3 (augD1, 93.50), ev3-d2-mwv33 (augD2, 98.67). Budget v2 on the older v32 harness: ev3-d1-mwv2 (98.83), ev3-d2-mwv2 (97.17). Time bank: multi-wave ev3-d1-mwb (94.17), ev3-d1-mwb2 (91.00), ev3-d2-mwb (91.33), ev3-d2-mwb2 (93.17), and one wave per 30-task shard ev3-d1-tc (96.67), ev3-d1-t86 (95.67). The two runs of mwv33 on augD1 differ by six points.

Tokens at first fit	<10k	10–20k	20–40k	40–80k	≥80k
Final harness, primary (352 fits)	31/33 (94%)	38/54 (70%)	80/105 (76%)	65/118 (55%)	12/42 (29%)
share of fits	9%	15%	30%	34%	12%
task finally solved	97%	85%	89%	75%	43%
Budget v2 family, primary (588)	44/48 (92%)	75/100 (75%)	131/170 (77%)	103/191 (54%)	19/79 (24%)
Time bank, primary (692)	38/43 (88%)	110/137 (80%)	158/212 (75%)	113/206 (55%)	16/94 (17%)
Final harness, tail chains (36)	–	–	3/4	6/11	6/21 (29%)
First *submit* instead of first fit, final (352)	14/14	34/38 (89%)	90/105 (86%)	93/132 (70%)	24/63 (38%)

Table 11. Share of first train-passing programs that are right on the test, by generated tokens before they were found. 95% Wilson intervals for the final harness: [80, 98], [57, 81], [67, 83], [46, 64], [17, 44].

Tokens (k)	<5	5–10	10–15	15–20	20–30	30–40	40–60	60–80	80–120	≥120
n	7	26	29	25	63	42	76	42	29	13
right	100%	92%	83%	56%	76%	76%	61%	45%	31%	23%

Table 12. Finer bins, final harness, primary chains. The dip at 15–20k appears in two of the three runs individually; we do not have an explanation for it.

Run	Harness	Score	Fits	Right	Median tokens	<10k	40–80k	≥80k
d1-mwv33	final	99.50	117	67%	33.4k	9/11	25/41	2/11
d1-mwv33r3	final	93.50	118	59%	36.1k	9/9	20/41	4/15
d2-mwv33	final	98.67	117	67%	35.3k	13/13	20/36	6/16
d1-mwv2	v2 on v32	98.83	119	66%	39.8k	7/7	22/38	6/21
d2-mwv2	v2 on v32	97.17	117	58%	32.8k	6/8	16/35	1/16
d1-mwb	time bank	94.17	113	66%	34.6k	5/5	19/37	1/13
d1-mwb2	time bank	91.00	116	61%	35.9k	6/7	21/34	3/17
d2-mwb	time bank	91.33	115	64%	31.3k	6/6	18/26	1/17
d2-mwb2	time bank	93.17	112	63%	37.4k	6/6	21/38	0/11
d1-tc	time bank	96.67	118	59%	34.8k	8/10	15/34	4/18
d1-t86	time bank	95.67	118	64%	35.9k	7/9	19/37	7/18

Table 13. Per run, primary chains. "Right" is the share of first fits that are right.

Wall time. The same fits binned by real time on our L40S machines (each lane runs at roughly 30–40 tokens per second there, while the budget clock assumes about 19) show the same shape: under 4 minutes 20 of 22 right (91%), 8–16 minutes 74% ($n = 102$), 16–32 minutes 60% ($n = 116$), 32 minutes or more 29% ($n = 56$). Wall time is not comparable across machines, so the main text states the result in tokens.

Caveats. All runs use two augmentations of the same 120 public tasks, so tasks repeat across runs, and absolute rates are inflated relative to hidden tasks. Difficulty confounds the curve: 52 of the 126 wrong first fits in the final runs still ended in a solved task.

I Token allocation: time bank versus budget v2

Run (augD1 unless noted)	Score	Tasks	Mean	Median	Max	At cap
d1-tc, time bank, one wave per shard	96.67	solved 92	63.3k	56.8k	188k	0
		not solved 28	121.9k	112.2k	241k	0
d1-mwb2, time bank, multi-wave	91.00	solved 87	60.1k	52.7k	194k	2
		not solved 33	111.0k	95.8k	279k	10
d2-mwb2, time bank, multi-wave (augD2)	93.17	solved 89	61.2k	62.7k	134k	2
		not solved 31	116.5k	98.0k	591k	8
d1-mwv33, budget v2	99.50	solved 94	61.7k	61.7k	190k	3
		not solved 26	106.6k	100.5k	192k	2
d2-mwv33, budget v2 (augD2)	98.67	solved 96	65.7k	62.5k	188k	1
		not solved 24	104.4k	93.9k	194k	2

Table 14. Generated tokens per task in the primary chain. "At cap" counts tasks that ended within 2k tokens of their final allotment. On d1-mwv33, 51 tasks were flagged hard and 19 tail chains used a further 1.69M tokens (8.58M in primary chains); on d2-mwv33, 53 tasks were flagged and 14 tail chains ran. Under the multi-wave time bank most cap hits are unsolved tasks, while single tasks could take up to 591k tokens.

J Fine-tuning

No submitted version uses fine-tuned weights. We tried supervised fine-tuning, DPO, on-policy distillation from a hinted teacher and GRPO, all on harness trajectories of tasks derived from public evaluation. Offline gains on the originals shrank on augmented copies (an SFT model gained about +7 on the originals it was trained on and about +3 on augmented copies), and no fine-tuned model beat the base model on the leaderboard.

Model	Offline	Leaderboard
SFT v1 (LoRA)	augD1 94.83 vs 88.83 (1 run); no gain on held-out hard tasks	–
DPO	augD1 78.83 vs 85.83	–
On-policy distillation, hinted teacher	hard pool 96.67 vs 102.5/104.5	–
SFT (full), shared expert mask	eval120 80.83/87.83 (a1/@2)	71.39, 63.47
Same, model's own expert mask	eval120 88.83/94.33; aug 82.33/87.83 and 83.0/87.5	67.36
GRPO from the SFT model	93.5/95.33 → 82.33/84.83 → 83.83/91.83, stopped	–
GRPO with self-distillation, iteration 6	eval120 up to 86.83/91.83 at iteration 2	63.89
Base model, same harness (v13, 4 submissions)	–	mean 69.2

Table 15. Fine-tuning attempts. At an equal-compute budget of 45k tokens on augD1 the order of the offline scores (SFT 82.33, GRPO iterations 75.33 and 74.17) matched the leaderboard order, the first sign that a Kaggle-equivalent augmented eval could predict the leaderboard. The SFT data also underfit: token accuracy on its own rows rose only 1–3 points.

K Leaderboard history

Date	Submission	What changed	Public LB
10-01	56759808 (v1)	unpruned, 2 lanes, INT4 MTP	22.92
10-02	56783614 (v2)	REAP-384 pruned at load time, 8 lanes	46.39
10-03	56806847	swap32-cut320, 87 configuration	60.42
10-05	56859795	review module v49 + watchdog	56
10-08–10	three submissions	v103	64.58, 67.64, 69.03
10-11	57053020	v104: FR-Spec, FP8 n-gram table, pre-pruned checkpoint	67.22
10-12	57118916	v106 (adds skipping attempt 2 when early and undoubted)	69.31
10-13	57152050	helper card	65.00
10-14	57192661 (v13)	v106 harness, pre-pruned cut320	71.81
10-15–25	three repeats of v13	–	68.75, 65.56, 70.83
10-17–19	fine-tuned models	SFT (shared mask)	66.81, 71.39, 63.47
10-20–21	57455219, 57495565	time-bank overdraft 15%	69.03, 65.56
10-23	57568600	SFT with its own expert mask	67.36
10-24	57598401	GRPO iteration 6	63.89
10-26	57680964 (v30)	32 lanes, FP8 KV, host KV tier, bf16 DeltaNet state	70.14
10-27	57714935 (v31)	24 lanes, INT4 KV, decode-cache fixes, sharded sampling	70.28
10-28–30	57750771 (v32) + 2 repeats	full serving stack, 32 lanes, 84 GiB host tier	70.97, 72.64, 66.81
10-31	**57850872 (v33)**	budget mechanism v2 + tail chains, token clock 86k	**73.89**
11-01	57885586 (v34)	48 lanes, FP8 indexer, memory-access fix	77.64 (after deadline)
11-02	57921706 (v34 repeat)	–	74.31 (after deadline)

Table 16. All scored public-leaderboard submissions of the Flash-Next agent (notebook `deepdreamer/fnext-w4a16-arc-submit`). Versions v1–v34 in the main text refer to the labelled rows. v33 is the submission for the Accuracy criterion.